RankCloak: Hiding Synthetic Cryptographic Artifacts in Plain English with Language Models

Alexander V. Mantzaris[1,*]

[1]School of Data, Mathematical, and Statistical Sciences, University of Central Florida, Orlando, FL, United States of America
[*]Correspondence: alexander.mantzaris@ucf.edu

ABSTRACT

RankCloak transcodes synthetic cryptographic artifacts into language-model next-token ranks and realizes those ranks as generated cover text. We compare direct subword representation with bounded ASCII-byte and hex-nibble codecs, including non-segmented generation and segmented forced spans followed by non-payload natural tails. The completed evaluation used 480 public deterministic artifacts from eight cryptographic classes, three open-source model families, and 18 English prompt templates; secondary recovery tests used Spanish and Simplified Chinese prompts. All 6,480 primary payload trials recovered the serialized payload under saved-token exact-copy replay (Wilson 95% confidence interval, 0.999408–1.000000), and all 576 multilingual trials recovered under the same condition. This contract is restrictive: visible-text detokenization and retokenization recovered 61.1%, normalization and formatting changes reduced recovery further, and markdown transfer, paraphrase, and cross-model decoding recovered none. Automated readability diagnostics were protocol-dependent and are not human ratings. Neural detectors strongly distinguished RankCloak covers from controls (matched ROC-AUC 0.990190 for TextCNN and 0.999834 for DeBERTa-v3-base), adverse evidence for concealment. RankCloak therefore provides a reproducible framework for measuring rank pressure, capacity, replay requirements, cover cost, and detectability, but does not establish a secure or deployment-ready covert channel.

Introduction

Steganography seeks to conceal the existence of communication, a goal distinct from the confidentiality supplied by encryption[1–4]. Text is an unusually constrained carrier because its syntax, semantics, and style are readily judged. Earlier systems used mimic functions, contextual substitution, and statistical language models to map secret symbols into plausible text[5,6]. Neural generators widened the choice space: recurrent and transformer language models can carry bits through controlled token selection, arithmetic coding, or related sampling rules[7–15].

Three questions are easily conflated in this literature. A payload representation may be invertible without producing economical cover text; a generated message may score well under its source model without surviving transmission; and a superficially readable message may remain statistically distinguishable from an ordinary control. Cryptographic artifacts sharpen these distinctions because their serializations have rigid syntax and high entropy. Simply placing such a string in prose exposes the artifact, whereas forcing a representation through a generator shifts the cost into token likelihood, length, replay state, and detectable distributional regularity. Those costs must be measured separately.

Next-token ranks provide another transferable representation. Given a shared language model and context, a sender can select the token at a prescribed rank; a receiver replaying the same context can recover that rank. Calgacus demonstrated rank transcoding between hidden and cover texts[16]. RankCloak addresses a narrower measurement problem: how should high-entropy, visibly patterned artifacts such as hashes, nonces, identifiers, ciphertexts, and signatures be represented as ranks, and what costs follow when those ranks are forced into generated cover text? This focus separates the underlying rank-transfer idea from the present contribution: explicit artifact codecs, deterministic replay rules, segmented forced spans, non-payload tails, and a common evaluation of capacity, cover length, quality diagnostics, recoverability, and detectability.

The measurement contribution is therefore not a claim to have originated next-token rank transfer. It is a controlled treatment of the complete path from an exact serialized artifact to codec ranks, forced cover tokens, replayed ranks, and recovered bytes, together with accounting for text that carries no payload. This path makes it possible to ask whether lowering the maximum permitted rank merely exchanges rank pressure for more positions, whether segmentation changes that exchange, and whether any apparent cover improvement persists when tails and detector performance are examined separately.

The representation is consequential. Directly tokenizing an artifact can be compact, yet its subwords may occupy extreme positions under a payload context. Transferring those ranks to a cover context can force improbable tokens. Fixed-radix byte and hexadecimal-nibble codecs instead bound the rank alphabet, at the cost of more forced positions. Segmentation divides that burden among multiple messages. A natural continuation can make each public message less abrupt, but it carries no payload and increases cover length. These mechanisms create measurable trade-offs rather than a guarantee that readers will perceive the output as natural language.

Recovery creates a second distinction. Rank identity is deterministic only when the receiver reproduces the model, tokenizer, prompt, filtering, prior tokens, and tie rule. Saved token identifiers provide the strongest exact-copy condition, but they are not an ordinary text-transmission channel. Detokenized visible text can tokenize differently even when it appears unchanged, and small edits can shift every later context[17]. A useful evaluation must therefore report exact-copy replay separately from visible-text transmission, controlled perturbations, and model mismatch rather than treating successful in-memory decoding as channel robustness.

Concealment must also be tested rather than inferred from the absence of visible hexadecimal or base64. Linguistic steganalysis has used convolutional, graph-neural, and few-shot classifiers[18–20]; generated-text detectors likewise show that distributional signals can be strong in-domain yet sensitive to attacks and distribution shift[21–23]. Related work has examined hidden communication among language-model agents and emerging steganographic capabilities[24,25]. RankCloak's defensive value is consequently an empirical question: can neural classifiers distinguish its outputs, across matched and held-out prompts, models, and codecs?

This defensive question does not convert neural steganalysis into the paper's primary contribution. Rather, detector performance tests a necessary claim boundary for the rank-transcoding method. A system that recovers exactly yet is easily classified can still illuminate the mechanics and failure modes of rank-based hiding, but it cannot be described as covert in the evaluated setting.

This revision evaluates those questions with completed rather than pilot evidence. It uses 480 deterministic outputs of declared cryptographic algorithms, three open-source 7B–8B model families, 18 English templates in six prompt categories, and secondary Spanish and Simplified Chinese templates. It measures exact serialized-payload recovery, grouped sensitivities, transformation failures and first divergence, automated readability and model-based quality, codec and protocol ablations, prompt-topic contrasts, formal capacity relationships, and computational overhead. Two neural architectures are evaluated on matched and held-out splits with leakage and near-duplicate sensitivity checks. The study does not include participants or deployment traffic. Its purpose is to establish a reproducible rank-transcoding measurement framework and to identify, without concealing adverse results, where that framework succeeds under controlled replay and where it fails as a candidate communication channel.

Methods

Deterministic ranks and payload representations

For autoregressive model (M), vocabulary $\mathscr{V}$, token history h, and logit $z_M(v \mid h)$, candidate tokens are ordered by decreasing logit and then increasing token identifier. The one-indexed rank is

$$\mathrm{rank}_h(v) = 1 + |\{u \in \mathscr{V} : z_M(u \mid h) > z_M(v \mid h) \text{ or } [z_M(u \mid h) = z_M(v \mid h) \wedge \mathrm{id}(u) < \mathrm{id}(v)]\}|. \quad (1)$$

The inverse $\mathrm{token}_h(r)$ selects the unique token at rank r. Token-ID tie-breaking is part of the shared configuration; a different rule can alter recovery.

The ordering is formed after any declared eligibility mask is applied. Because the softmax transformation preserves logit order, ranks can be computed from filtered logits without sampling probabilities, but the mask itself must be identical at both ends. Model weights, tokenizer vocabulary and normalization, chat-template rendering, quantization, backend behavior, prompt bytes, prior token IDs, and the tie rule jointly define the replay state. RankCloak supplies no synchronization mechanism for those objects.

Direct subword representation tokenizes displayed payload x under a payload-side context and records each payload token's rank. This preserves a direct rank-transcoding interpretation and may use few symbols, but supplies no useful upper bound below the vocabulary size. The bounded codecs operate on the serialized artifact instead. For ASCII-byte base B, each byte $b \in [0, 255]$ is represented by $w = \lceil \log_B 256 \rceil$ fixed-width base-B digits d_j, then mapped to ranks $d_j + 1$. The evaluated $B = 8$ and $B = 16$ versions therefore use ranks 1–8 and 1–16. Hex-nibble coding maps each canonical lowercase hexadecimal character to its value 0–15 and then to ranks 1–16. Fixed width and the serialized length make decoding unambiguous; the hex codec applies only to eligible hex-like payloads. Full codec equations and pseudocode are in Supplementary Note S1.

The decoder subtracts one from each bounded rank, rejects digits outside the declared base, groups the fixed-width sequence, and reconstructs the exact byte serialization. Hex decoding likewise rejects ranks outside 1–16 and inverts the canonical nibble map. Expected byte length, syntax, and SHA-256 digest are checked after inversion. These checks distinguish rank equality, representation equality, and serialized-payload equality; only the last is the primary recovery endpoint. Base64 and UUID display forms are not silently passed through the hex codec.

Cover encoding, segmentation, and tails

For rank sequence $R = (r_1, \dots, r_n)$, cover prompt p, and initial token history $h_1 = \text{tokenize}(p)$, non-segmented encoding emits $y_i = \text{token}_{h_i}(r_i)$ and updates $h_{i+1} = h_i \| y_i$. Decoding replays the same prompt and tokens, obtains $\hat{r}_i = \text{rank}_{h_i}(y_i)$, and applies the inverse payload codec. The procedure is deterministic conditional on the complete shared configuration.

The retained record stores the generated IDs, codec and payload identities, prompt/configuration identities, and forced-position metadata required by the declared replay mode. Non-segmented decoding consumes the complete payload-bearing token sequence. It does not infer a payload boundary from prose, and it does not search over alternative prompts or tokenizations. Any such search would define a different channel and was not evaluated.

Segmented encoding partitions R into ordered chunks. Each chunk is generated under its declared prompt template as a payload-bearing forced span. The canonical segmented configuration uses eight ranks per chunk, no lead-in, a deterministic safe-text filter, and dynamic greedy completion. After the forced positions, greedy tokens are appended until at least eight natural tokens have been produced and an allowed semantic-completion condition is met: terminal punctuation or a double newline, balanced delimiters, and no declared dangling connective or punctuation. An emergency cap of 256 natural tokens prevents indefinite generation and is recorded as censoring. Alternative fixed-tail and no-tail policies were evaluated. Natural tails carry no payload ranks and are ignored by the decoder; reported full-message length and quality therefore mix a payload-bearing forced span with a non-payload continuation.

Segments retain their order, half-open rank slice, forced-token offsets, and role mask. Decoding reinitializes each declared segment prompt, replays only the forced span, concatenates recovered ranks in segment order, and ignores every tail token. Optional greedy lead-ins also carry no rank, but unlike tails they precede the forced span and therefore alter every subsequent conditional distribution; their length and token sequence must be reproduced. Lead-in lengths and segment sizes were varied in the ablation matrix.

Filtering is applied deterministically before rank selection. The canonical safe-text rule removes declared token forms that cannot be used consistently as public cover choices; sender and receiver must apply the identical ordered mask. The ablation compares this rule with no filter and a roundtrip-stable rule. Filtering changes the candidate rank order and is not a post-generation repair.

The filter is constructed before payload-rank lookup rather than used to rewrite emitted text. Consequently, a token removed at one endpoint shifts the indices of all lower-ranked eligible choices. The Mistral roundtrip-stable-filter vocabulary was empty under the isolated frozen criterion; its corresponding work units were declared unavailable rather than decoded as failures.

Replay contract and transmission tests

The strongest recovery endpoint replays saved token IDs with the same model file, tokenizer, quantization, prompt template, payload codec, filter, segmentation metadata, and deterministic rank rule. Saved token IDs are an exact-copy record, not an ordinary text channel. Visible cover strings were separately detokenized and retokenized because an apparently unchanged string can yield another token sequence. The transformation matrix also tested raw unmodified text, final-tail truncation, Unicode NFKC, quote conversion, whitespace trimming and collapse, line-ending conversion, character deletion/insertion/substitution, token deletion, a markdown blockquote copy/paste proxy, paraphrase, and cross-model decoding. Limited canonicalization cells applied declared transformations before replay. First-divergence categories describe where decoded traces first differ; they are diagnostics, not causal proof. Supplementary Note S4 inventories direct, partial, and untested reviewer-requested perturbations.

Transformations were frozen as separate conditions, not composed into an uncontrolled edit stream. Raw transmission applies the transformed visible text directly to the declared tokenizer and replay logic. The limited canonicalization pipeline applies its specified normalization before that replay; it is not a learned error corrector. Cross-model outcomes use a different declared model/tokenizer pair and are summarized per source cover so two alternative decoders are not mistaken for two independently generated messages. The first recorded mismatch includes segment and forced-position indices, expected and observed token/rank information where available, and whether a rank exceeded its codec bound.

Capacity, rate, length, and quality definitions

Let payload information content be H bits and let a bounded rank alphabet contain B symbols. The minimum fixed-radix forced length and nominal rate are

$$n_B = \left\lceil \frac{H}{\log_2 B} \right\rceil, \qquad R_B = \frac{H}{n_B}. \tag{2}$$

For n_{forced} payload positions and n_{tail} non-payload positions, effective full-message rate is

$$R_{\text{eff}} = \frac{H}{n_{\text{forced}} + n_{\text{tail}}}. \tag{3}$$

The cover-length residual is observed full length minus the codec-implied forced length. Positive residuals represent tails or other cover overhead, not extra payload capacity. Forced-span quality is summarized by mean model log probability $Q = -n^{-1}\sum_i[-\log p_M(y_i \mid h_i)]$ with sign conventions stated per table; rank-bounded comparisons are made under identical contexts. These probabilities and the automated diagnostics below are not human judgments. Supplementary Note S8 gives assumptions, proof sketches, residual definitions, and the retained-trace limitation.

For an m-byte ASCII codec, the implemented forced count is exactly $m\lceil\log_B 256\rceil$; hexadecimal uses two nibbles per serialized byte. Segmentation preserves the sum of forced positions while changing the number of resets and potential tails. A bounded rank $r \le B$ also places the selected token's negative log probability between those of the rank-1 and rank-B eligible tokens under the same history. This is a conditional model-likelihood bound, not a statement about grammar, meaning, or reader judgment.

Corpus, models, prompts, and experimental matrices

The corpus contains 480 public deterministic test vectors: 60 each of SHA-256 hexadecimal digests, HMAC-SHA256 hexadecimal tags, deterministic 96-bit nonces, 128-bit hexadecimal tokens, UUIDv4 values, AES-256-GCM base64 outputs, ChaCha20–Poly1305 base64 outputs, and Ed25519 base64 signatures[26–32]. They are algorithm-defined artifacts for reproducible testing, not operational secrets or deployment samples.

Declared seeds and generation procedures make every artifact public and reproducible while preserving the syntax and length of the named algorithm's output. Authenticated-encryption and signature rows are genuine deterministic test outputs of the declared algorithms, not random strings relabeled as ciphertext or signatures. This design tests realistic serialization distributions without handling private keys, credentials, or operational traffic.

Generation used Meta-Llama-3-8B-Instruct Q4_K_M, Qwen2.5-7B-Instruct Q4_K_M, and Mistral-7B-Instruct-v0.3 Q4_K_M. Eighteen English prompt templates cover six categories: casual conversation, professional communication, forum question-and-answer, recipe/procedure, personal narrative/blog, and factual/explanatory writing. The primary matrix comprises 14,400 generated work units: 6,480 RankCloak payload trials and 7,920 ordinary-generation controls. Secondary matrices cover ablations, robustness, multilingual recovery, held-out evaluation, and detector fitting. Declared combinations lacking a compatible retained output are unavailable rather than failed. The complete ledger contains 38,504 reconciled units.

All three models used their embedded GGUF tokenizers and the same one-model-at-a-time rank-replay harness. The 18 prompts were fixed before the completed primary matrix; multi-topic segmentation rotates across the schedule, while single-topic segmentation retains its declared topic. Ordinary controls were generated for the matched prompt/model/representation design rather than drawn from an unrelated corpus. Corpus construction, model identities, prompt categories, and work accounting are detailed in Supplementary Note S2 and Supplementary Tables S1–S4.

The Spanish and Simplified Chinese experiments are secondary exact-copy recovery tests, not evaluations of human-perceived multilingual naturalness. Exact model paths, prompt text, hashes, matrix dimensions, and the 432 unavailable units are reported in Supplementary Note S2. The configured GGUF files are absent from the current checkout. Consequently, fresh generation replay is unavailable here even though retained outputs are bound to model and configuration manifests.

Recovery and robustness endpoints

The confirmatory endpoint is exact equality of the decoded serialized payload to its declared source after saved-token replay. Trial-level Wilson 95% confidence intervals are reported[33]. Strict sensitivities require success for every trial within each payload and within each model–payload group, avoiding treatment of repeated prompt/protocol trials as independent evidence. Multilingual recovery is secondary. Transformation recovery is descriptive over the retained robustness covers. Unavailable cells are excluded from denominators and shown separately.

The trial endpoint answers whether one generated record recovers. The payload-group endpoint passes only if every model, prompt, and eligible protocol trial for that artifact succeeds; the model–payload endpoint applies the same rule within a model. These strict summaries guard against inflating precision by repeated conditions. Because every primary trial succeeded, a recovery generalized linear mixed model was not estimable under complete separation; the declared Wilson sensitivity endpoints were reported instead.

Automated cover diagnostics

A balanced set of 504 computational stimuli was selected across seven generation conditions and 18 prompt templates. Surface diagnostics included Flesch Reading Ease[34], declared artifact flags, and TF–IDF similarity to the prompt; intervals were bootstrapped over prompt templates. These are automated readability diagnostics, not participant ratings. Model-based quality compared source-model log probability and a held-out evaluator's log probability across protocols, retaining forced-span/full-message distinctions. No ethics-authorized recruitment or survey deployment occurred: there were zero participants and zero human ratings. Prepared study materials are planning artifacts only.

The seven conditions were balanced at 72 stimuli each, and resampling over prompt templates reflects prompt-level variation rather than treating all strings as independent. Surface flags count frozen formatting/artifact patterns. TF–IDF similarity is lexical, and Flesch is an English syllable/sentence heuristic. Model-based measures are reported both on payload-bearing forced spans and on full messages that include greedy tails; the latter cannot be used to infer that the forced tokens improved.

Neural detector evaluation

The frozen detector corpus contains 15,840 rows, balanced between 7,920 RankCloak covers and 7,920 matched controls and grouped by 480 payloads. A TextCNN-equivalent classifier[18,35] used lowercase tokens, a 30,000-term vocabulary, length 256, 300-dimensional embeddings, 100 filters each at widths 3 and 5, dropout 0.5, and Adam learning rate 0.001. A DeBERTa-v3-base classifier used length 256, learning rate 2×10^{-5}, weight decay 0.01, and three epochs, building on disentangled-attention language representations[36]. Payload groups, not individual rows, defined splits.

Each architecture was fit on one matched split, 18 held-out-template splits, three leave-one-model splits, and six leave-one-codec splits: 56 completed fits and 55,440 predictions. Primary metrics were ROC-AUC and balanced accuracy; payload-group bootstrap intervals used 2,000 resamples[37]. Precision, Brier score, and sensitivity at 1% false-positive rate were exploratory. Audits tested exact train/test text and token-sequence identity, then diagnosed normalized character n-gram near-duplicates at a declared threshold. A sensitivity analysis excluded affected test payload groups without retraining. Fixed-seed repeats assessed same-device CUDA repeatability. CPU neural training was infeasible and was not used to infer CPU/GPU equivalence.

Held-out-template splits remove one prompt template from training, leave-one-model splits remove a source-model family, and leave-one-codec splits remove one representation/protocol family. Ranges across these heterogeneous splits are reported as minima and maxima rather than pooled confidence intervals. Exact leakage checks and lexical similarity serve different purposes: passing identity checks does not rule out threshold-defined near duplicates. Exclusion sensitivity removes affected test payload groups only and therefore cannot retroactively repair a frozen training design.

Statistical inference and overhead

Inference respected payload and prompt-template grouping; segments were never analyzed as independent experimental units. Wilson intervals summarize all-success recovery. Prompt-template and payload-group bootstraps used 2,000 resamples. Paired ablations use payload as the bootstrap and pairing unit. Mixed models included payload and prompt grouping where supported, with negative-binomial modeling for overdispersed artifact counts and Gaussian models for continuous outcomes[38,39]. Holm adjustment controls selected contrast families; Benjamini–Hochberg values accompany exploratory families. Three of five fitted mixed models—effective rate, source-model cover log probability, and held-out-evaluator log probability—were singular. No fixed-effects fallback was substituted. Compact ablation, topic, auxiliary detector, and near-duplicate analyses are exploratory or post-outcome where labeled.

Model, protocol, prompt category, and payload class enter the locked outcome models as appropriate, with payload and prompt random intercepts where supported and an exposure offset for artifact counts. Convergence, dispersion, zero inflation, singularity, and residual diagnostics were retained rather than filtered by result direction. Selected model/protocol contrasts use Holm correction; broader exploratory families retain their declared multiplicity labels. The compact ablation and topic tables are post-outcome extractions from frozen outputs and do not become confirmatory merely because an interval excludes zero.

Generation overhead uses retained inclusive wrapper wall time, encompassing setup and decoding work recorded by the harness. Payload throughput is payload bits divided by generation time; effective rate follows Eq. 3. Detector benchmarks report fixed CUDA-device wall/training time and peak allocated VRAM. CPU time, repeated warm-up measurements, and broad hardware generalization are unavailable.

Results

Expanded evaluation and exact-copy recovery

The completed primary design produced 14,400 work units across three model families and 18 English prompt templates. Of these, 6,480 were payload-bearing RankCloak trials spanning the six protocol/codec configurations and 7,920 were ordinary controls. All 6,480 decoded serialized payloads exactly under saved-token replay (1.000000; Wilson 95% CI, 0.999408–1.000000; Table 1). The stricter aggregation also passed for all 480 payload groups (CI, 0.992061–1.000000) and all 1,440 model–payload groups (CI, 0.997339–1.000000). These grouped endpoints prevent repeated prompts and protocols from inflating the apparent number of independently generated artifacts.

The recovery question was whether codec inversion remained exact after forcing the ranks into cover contexts, rather than whether emitted ranks could merely be read back from the same loop. Success was defined at the original serialization and digest, and every model and eligible protocol stratum reached that endpoint. The all-success result therefore supports deterministic implementation fidelity across the evaluated prompt/model matrix. Its uncertainty is one-sided in substance:

the data bound the rate of error under this completed design, but cannot show that an unobserved replay environment or transmission channel would also be error-free.

Secondary exact-copy tests recovered 288/288 Spanish trials and 288/288 Simplified Chinese trials, or 576/576 in total. They establish recovery for those retained prompts under the same saved-token contract; they do not establish multilingual naturalness or robustness to ordinary text transfer. Across primary, ablation, robustness, multilingual, evaluator, and detector work, the final ledger reconciled 38,504/38,504 units: 38,072 succeeded and 432 were declared unavailable. No unit remained and none was classified as an execution failure. Recovery strata and secondary-language endpoints appear in Supplementary Tables S5–S6.

Design or endpoint	Scope	Result
Payload corpus	Eight algorithm-defined classes, 60 public deterministic artifacts per class	480 artifacts
Model and prompt coverage	Three open-source 7B–8B families; 18 English templates in six categories	14,400 primary work units (6,480 payload; 7,920 control)
Primary exact-copy recovery	Serialized payload, saved-token replay	6,480/6,480; 1.000000 (95% CI 0.999408–1.000000)
Strict grouped sensitivity	Payload groups; model–payload groups	480/480 (CI 0.992061–1.000000); 1,440/1,440 (CI 0.997339–1.000000)
Spanish secondary recovery	Saved-token replay	288/288 (CI 0.986837–1.000000)
Simplified Chinese secondary recovery	Saved-token replay	288/288 (CI 0.986837–1.000000)
Complete work ledger	All declared revision matrices	38,504/38,504 reconciled: 38,072 success; 432 unavailable; 0 execution failure

Table 1. Evaluation design and recovery endpoints. Confidence intervals are trial- or group-level Wilson 95% intervals as identified. Recovery requires saved-token exact-copy replay with the declared shared configuration. Unavailable work units are configuration/retained-output absences, not observed decoding failures.

Capacity, rank bounds, and cover-length overhead

Direct subword representation generally required fewer payload symbols, but its ranks were not bounded below the vocabulary size. ASCII-byte and hex-nibble representations increased forced length while restricting choices to ranks 1–8 or 1–16. This is the intended capacity trade-off: a smaller rank alphabet reduces per-position payload capacity and increases length, rather than providing free improvement in cover quality.

Figure 1 asks whether the recorded forced positions follow the codec-implied requirement and where additional cover length enters. For a fixed payload, moving from direct token ranks to a bounded alphabet controls the worst permitted rank at the cost of an algebraically longer sequence. Segmentation does not change the payload rank count; it changes how that sequence is divided among contexts and how often a non-payload continuation can be added. Effective full-message rate therefore falls when tails grow even though forced-span capacity is unchanged.

The retained theory audit covered 17,424 records; 7,008 contained all fields required for both capacity and quality-bound checks (Fig. 1). There were zero forced-position residuals, zero rate-bound violations, and zero evaluated quality-bound violations. There were 2,976 positive cover-length residuals, attributable to tails or other non-payload cover overhead. These checks confirm the local algebra and logged invariants of Eqs. 2–3. They do not validate human naturalness, and the retained record structure does not expose a complete encoder–decoder cascade trace; that stronger proposition was therefore not directly evaluable (Supplementary Table S14).

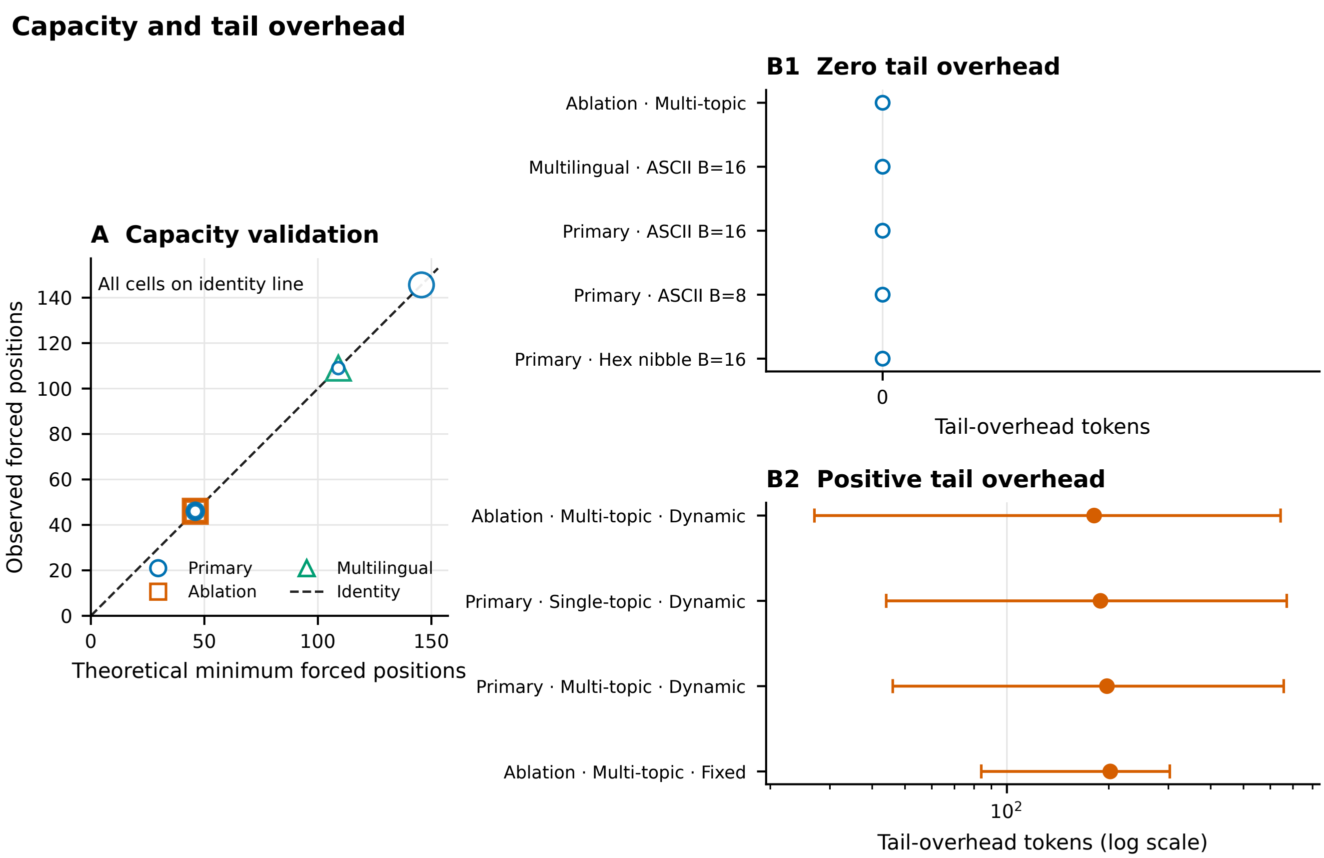

Figure 1. Capacity and tail overhead. Fixed-radix payload length and effective full-message rate are checked against their declared algebraic bounds, while the cover-length residual isolates tokens beyond the forced payload positions. Across 17,424 retained records, 7,008 were evaluable for the complete plotted capacity/quality checks; no forced-position, rate-bound, or evaluated quality-bound violation occurred. Positive residuals in 2,976 records represent natural tails or other cover overhead. Empirical agreement with these logged relationships is not evidence of human-perceived naturalness, and the complete encoder–decoder cascade cannot be reconstructed from the retained trace schema.

Exact-copy replay and transmission fragility

Replay mode materially changed recovery (Fig. 2). Saved-token replay recovered 144/144 robustness covers, whereas detokenizing the same covers and retokenizing their visible text recovered 88/144 (0.611111). Thus, even an unedited visible string did not guarantee the original token sequence. Raw unmodified transmission recovered 144/144, but common transformations were damaging: Unicode NFKC recovered 82/144 (0.569444), quote conversion 56/144 (0.388889), and whitespace trimming 28/144 (0.194444). Markdown copy/paste and paraphrase recovered 0/144, and the two cross-model decoder outcomes for each cover recovered 0/288.

These conditions address a different question from the primary result: whether a cover can leave its saved-token record and still recreate the forced ranks. The answer depended sharply on the frozen channel definition. The saved-ID, visible-retokenization, and raw-transmission rows are distinct replay procedures and should not be pooled. Their contrast shows why the receiver contract must name the tokenization path, not merely say that the visible text was copied exactly. Cross-model zero recovery further shows that similar prompt intent does not substitute for model and tokenizer identity.

Deleting only the declared final 10% of a message recovered 144/144 because that operation was constrained to non-payload tail tokens. It is not evidence of arbitrary truncation tolerance: removing a forced token or earlier context would change the decoded sequence. The broader matrix also showed low recovery after token or character edits and line-ending conversion (Supplementary Note S4). Limited canonicalization preserved unmodified controls but did not recover the available transformed cells, so it did not demonstrate a general mitigation.

Tail-only truncation is consequently a boundary check on the decoder definition rather than a successful error-correction result. The decoder is designed to ignore tails, so removing only tail labels should be harmless. By contrast, prefix changes and edits inside a forced span can alter a segment's initial tokenization or model history and propagate. The failed limited-canonicalization cells show that normalizing a small declared set after the fact was insufficient in this matrix; no broader mitigation is inferred.

Relative to the reviewer-requested inventory, five transformation classes were tested directly, three had only partial proxies, and three were untested. In particular, arbitrary prefix/suffix insertion, general case conversion, and general punctuation edits were not tested; line wrapping, email/markdown quoting, and sentence-boundary changes were only partially represented. First-divergence labels locate the first observed mismatch among tokenization, context, rank, or payload stages, but are descriptive and cannot by themselves establish why a transformation failed (Supplementary Fig. S1 and Supplementary Tables S7–S8).

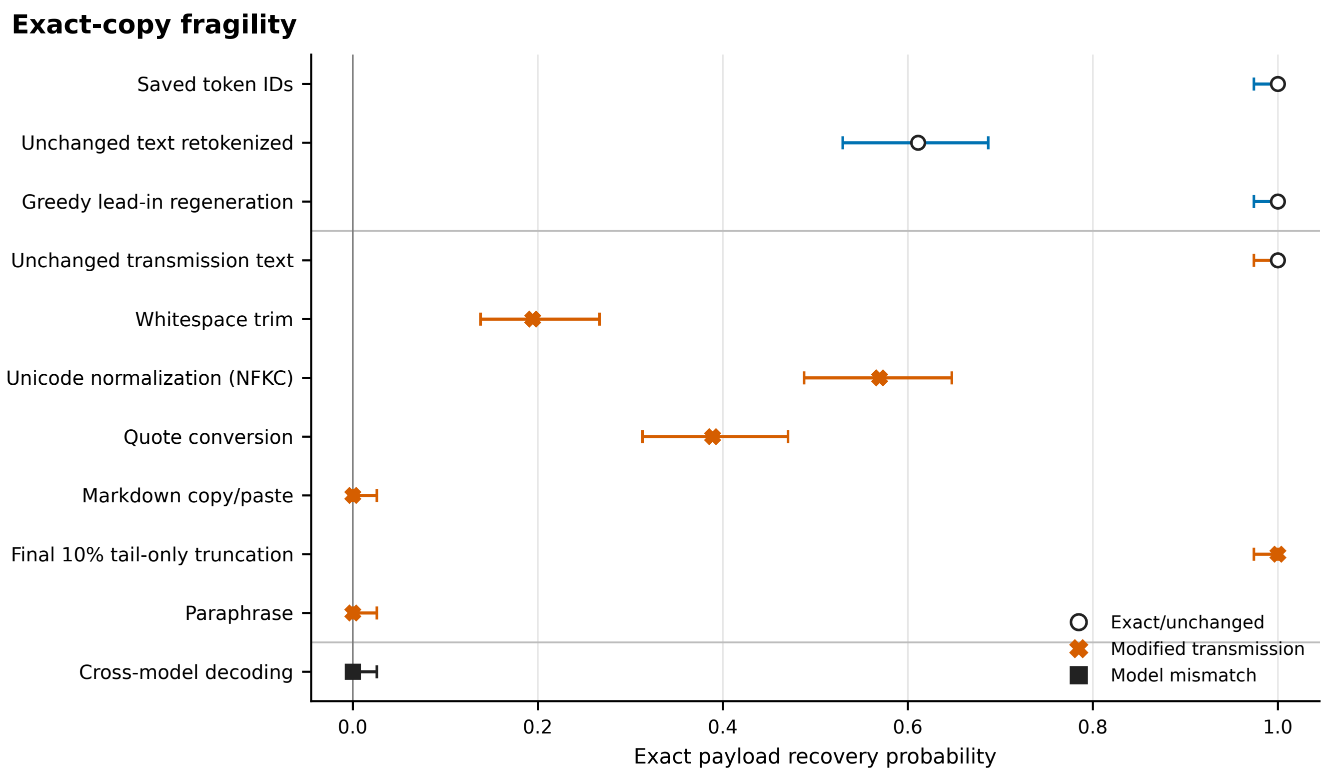

Figure 2. Exact-copy fragility. Recovery proportions and Wilson 95% intervals are shown for the principal replay and transmission conditions. Saved token IDs preserve the strongest replay contract; visible-text detokenization/retokenization already reduces recovery. NFKC normalization, quote conversion, and trimming reduce it further, while markdown copy/paste, paraphrase, and cross-model decoding yield no successful payloads. Final-10%-tail truncation removes only declared non-payload tail tokens and must not be interpreted as arbitrary truncation tolerance. The full transformation matrix, denominators, limited canonicalization cells, and first-divergence diagnostics appear in Supplementary Figure S1 and Supplementary Note S4.

Automated readability and model-based cover quality

The balanced computational set contained 504 stimuli across seven conditions and 18 prompt templates. Ordinary controls had mean Flesch Reading Ease 61.940597 (prompt-template bootstrap 95% CI, 54.897042–68.143167), and segmented full messages had mean 60.551834 (CI, 56.873661–64.135677; Fig. 3). The intervals overlap substantially. Surface flags and prompt similarity supplied additional diagnostics, but none is a human rating.

The principal question was whether obvious surface diagnostics changed when bounded ranks and segmentation were used, while preserving prompt-level uncertainty. At this aggregate level, the two highlighted Flesch intervals do not show a clear separation. Flesch is driven by word, sentence, and syllable counts, however, and can assign similar scores to passages with very different coherence. Surface flags detect only declared patterns, while prompt similarity is a bag-of-words comparison. Agreement among these quantities would still not establish reader acceptance.

Source-model and held-out-evaluator log-probability contrasts did not support a universal direction of cover quality. Forty-two of 45 contrasts in each evaluator family had Holm-adjusted $p < 0.05$, yet estimates changed sign across protocol/model cells: source-model contrasts ranged from -5.298565 to +2.684536 and held-out-evaluator contrasts from -6.161972 to +3.287497. Both quality mixed models were singular, so these conditional model-scale differences require caution. The forced span also answers a different question from the full message: only the former carries ranks, while the latter includes a sometimes long greedy tail.

The large number of adjusted contrasts excluding zero therefore does not yield a single quality ranking. It instead shows protocol-by-model dependence: a cover family can score higher in one conditional evaluator cell and lower in another. Full-message scores may also be dominated by greedy tokens that the decoder discards. Reporting the forced span separately prevents fluent tail text from obscuring likelihood pressure at payload-bearing positions, but neither scale has been calibrated to human judgments.

There were zero participants and zero human ratings. Established automated metrics and prepared study materials therefore only partially address the request for human evaluation. Participant research remains uncollected and would require ethics authorization, recruitment, and deployment; the planning material is not treated as an administered survey or a result (Supplementary Table S9).

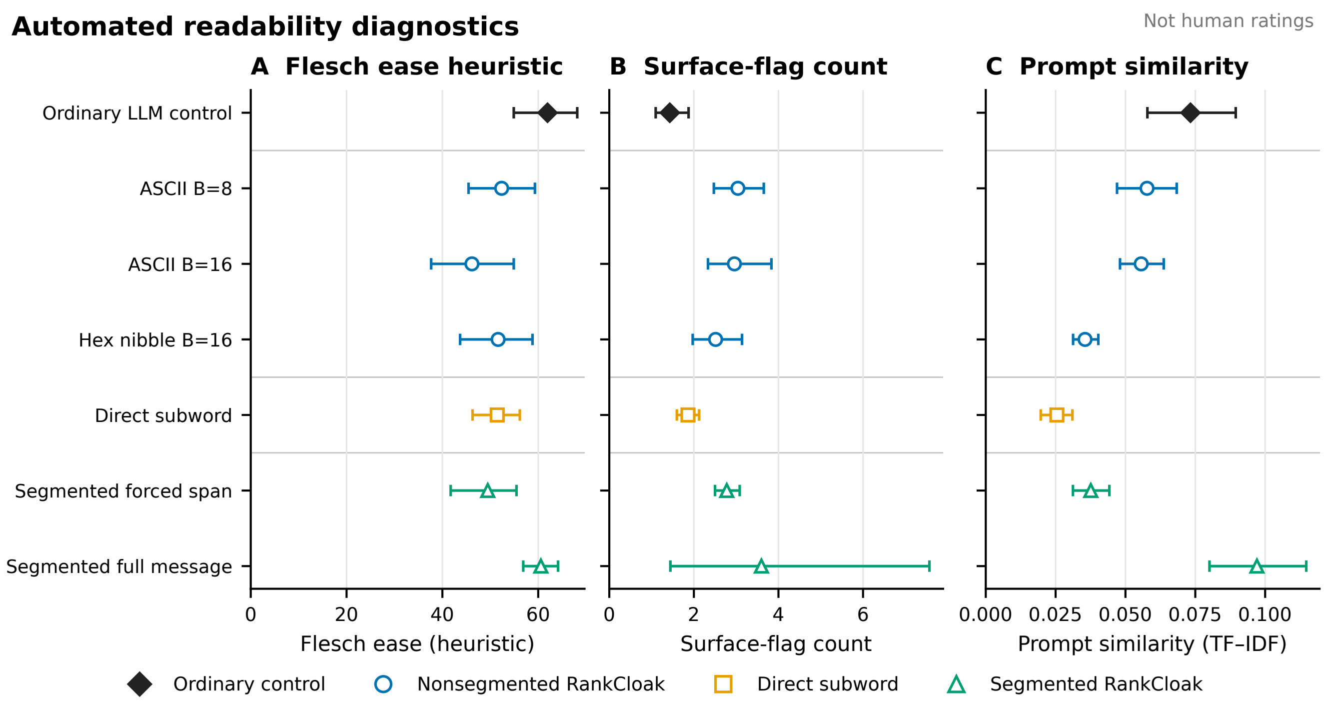

Figure 3. Automated readability diagnostics. Flesch Reading Ease and related surface diagnostics are summarized over 504 balanced computational stimuli, with uncertainty resampled over prompt templates. Segmented full messages include both payload-bearing forced spans and non-payload natural tails. These measures describe surface properties of the retained text and are not human ratings, evidence of human-perceived naturalness, or substitutes for a participant study. No participant was recruited and no rating was collected. Model-based quality analyses and their singular-fit warnings are reported separately in the text and Supplementary Note S5.

Ablations, dynamic stopping, and prompt-topic effects

The locked compact ablation extraction compared 48 paired payload groups across three models and is exploratory/post-outcome (Table 2). A 32-token lead-in, relative to no lead-in, reduced mean log probability by 1.652327 (95% CI, -1.749091 to -1.553103) and added 159.590278 full-message tokens (CI, 125.829861–191.030035). This adverse result is consistent with added context and boundary cost; the contrast does not prove a single failure mechanism. Increasing the segment size from 8 to 32 ranks increased effective rate by 1.231461 bits per full token (CI, 1.073116–1.388188) and reduced full length by 174.500000 tokens (CI, -208.405035 to -140.573264), trading fewer tails for longer forced spans.

The lead-in and segment results locate different costs. A lead-in precedes the forced span, consumes visible tokens, and changes the distribution used for every encoded rank; it therefore cannot be treated as a harmless stylistic prefix. Larger segments keep the forced count fixed but require fewer prompt resets and tail opportunities. Their higher effective rate is consistent with reduced overhead, while the forced span itself lasts longer before an ordinary greedy continuation begins.

Removing the safe-text filter changed mean log probability by approximately -0.0018 (canonical CSV estimate -0.001797; CI, -0.037738–0.035394) and effective rate by -0.062969 bits per full token (CI, -0.134265–0.007651); both intervals included zero. The Mistral roundtrip-stable-filter cell was unavailable for 48 work units, although the no-filter contrast included all three models. The filter results therefore do not establish a quality benefit.

The near-zero selected filter estimates are important null evidence. The deterministic filter remains necessary to define an agreed candidate set for the canonical protocol, but this ablation did not show that it improved the selected likelihood or rate endpoints. Nor can the unavailable stable-filter Mistral condition be treated as evidence for or against that condition; it is an absence caused by the frozen isolated-vocabulary criterion.

Compared with dynamic stopping, no tail increased effective rate by 3.171551 bits per full token and shortened messages by 254.951389 tokens. This arithmetic consequence of removing non-payload text is not evidence of better cover quality. A fixed 20–60-token sentence-tail policy shortened messages by 65.708333 tokens relative to dynamic stopping, whereas its selected effective-rate interval included zero. Dynamic stopping remains a declared heuristic with an emergency cap, not a semantic judgment by readers.

The tail ablation also prevents a misleading optimization conclusion. Maximizing bits per full token alone selects no tail because every added non-payload token enlarges the denominator. RankCloak introduced tails for a different, unverified purpose: allowing a forced fragment to continue to a rule-defined stopping point. Without human ratings, the experiment measures the length/rate price of that choice but cannot determine whether the price purchases perceived naturalness.

Across 15 locked topic contrasts, one Llama artifact-count comparison excluded the null: multi-topic versus single-topic expected count ratio 1.276361 (95% CI, 1.114743–1.461409; Holm-adjusted $p = 0.003294$). The other 14 adjusted p-values were at least 0.396248. This compact extraction was performed post-outcome without refitting models and is exploratory; it does not support a general prompt-topic advantage.

The solitary non-null row concerns one model and one artifact-count outcome. It did not recur across the other model/outcome combinations after adjustment. Accordingly, prompt diversity is retained as design coverage rather than promoted as a performance-enhancing intervention. Reviewer-relevant ablations and all topic contrasts appear in Supplementary Fig. S2 and Supplementary Tables S10–S11; the complete 60-row contrast table remains machine-readable in the public repository.

Analysis	Sample/group	Principal estimate	Uncertainty or adjusted test	Classification	Main limitation
32-token lead-in vs none	48 paired payloads; 3 models	Mean log probability -1.652327; full length +159.590278	CIs -1.749091 to -1.553103; 125.829861 to 191.030035	Exploratory, post-outcome	Boundary/context interpretation is not causal
32- vs 8-rank segments	Same	Effective rate +1.231461 bits/full token; length -174.500000	CIs 1.073116–1.388188; -208.405035 to -140.573264	Exploratory, post-outcome	Fewer tails accompany longer forced spans
No filter vs safe filter	Same	Log probability -0.001797; rate -0.062969	CIs -0.037738–0.035394; -0.134265–0.007651	Exploratory, post-outcome	Both selected intervals include zero; stable-filter Mistral cell unavailable
Tail policy	Same	No tail: rate +3.171551, length -254.951389; fixed tail: length -65.708333	Selected 95% CIs in Supplementary Note S6	Exploratory, post-outcome	Rate gain from deleting non-payload text is not quality gain
Topic schedule	15 locked contrasts	Llama artifact-count ratio 1.276361	CI 1.114743–1.461409; Holm $p = 0.003294$; other $p_{\mathrm{Holm}} \geq 0.396248$	Exploratory, post-outcome	One selected model/outcome contrast excludes null
Generation overhead	18 model–protocol cells; 12,144 matched rows	Mean 1.674368–16.087628 s; 13.631855–100.851772 bit/s	Payload-bootstrap intervals in Supplementary Note S9	Supporting measurement	Inclusive wall time; no CPU time or warm-up repeats

Table 2. Compact reviewer-requested analysis and operational summary. Differences are level minus canonical unless otherwise stated. Ablation and topic rows are a locked exploratory/post-outcome extraction from retained analyses; no new model was fit. Reviewer-relevant ablations, complete topic rows, availability, and timing cells are in the Supplementary Information; the full 60-row contrast table remains machine-readable in the repository.

Neural steganalysis

Both classifiers strongly separated RankCloak covers from matched controls (Fig. 4). TextCNN matched ROC-AUC was 0.990190 (payload-group bootstrap 95% CI, 0.987740–0.992682) with balanced accuracy 0.951010 (CI, 0.943456–0.958466). DeBERTa-v3-base ROC-AUC was 0.999834 (CI, 0.999696–0.999939) with balanced accuracy 0.991919 (CI, 0.987654–0.995665). High performance is adverse evidence for concealment, not a security feature of RankCloak.

This test directly challenges the premise that removing visible artifact syntax is sufficient for concealment. The corpus was balanced between RankCloak and matched ordinary controls and split by payload group, so the matched estimates cannot be attributed to class imbalance or row-level payload leakage. Both a compact convolutional model and a pretrained transformer found strong signal. The evidence is limited to this constructed corpus and training procedure, but within that scope it contradicts any claim of practical undetectability.

Generalization depended on architecture and split. Across 18 held-out templates, TextCNN ROC-AUC ranged 0.720421–0.981705 and balanced accuracy 0.565909–0.919318; DeBERTa ranges were 0.996291–0.999979 and 0.952273–0.995455.

Leave-one-model ROC-AUC ranges were 0.925623–0.973650 for TextCNN and 0.996772–0.999979 for DeBERTa; leave-one-codec ranges were 0.966532–0.998634 and 0.997917–1.000000. These are ranges across distinct held-out splits, not confidence intervals for one pooled estimate.

The TextCNN template range shows meaningful distribution sensitivity: a high matched score alone does not specify behavior for every new prompt. DeBERTa remained near perfect over the evaluated held-out regimes, which strengthens the adverse conclusion for these model, prompt, and codec families. It still does not establish performance on arbitrary human text, unseen generation systems, or future RankCloak modifications. The ranges deliberately preserve heterogeneity instead of averaging unlike test sets.

Exact text-identity and token-sequence leakage checks passed for all 28 frozen splits. A stricter lexical diagnostic nevertheless found 53 normalized near-duplicate pairs at the declared threshold, including 44 cross-payload pairs, affecting 16 splits. Excluding affected test payload groups without retraining changed ROC-AUC by -0.001925 to +0.004568 for TextCNN and -0.000038 to +0.000166 for DeBERTa; balanced-accuracy changes were similarly small. This sensitivity does not remove the design limitation because it is diagnostic, threshold-dependent, and does not rebuild the training sets. Fixed-seed repetition on the same CUDA device reproduced scores exactly for both architectures. It supplies same-device repeatability only; no CPU/GPU equivalence experiment was completed.

Small exclusion deltas show that the observed metrics were not numerically overturned by this particular thresholded removal, but they cannot prove independence of lexical structure. The sensitivity discarded affected test payload groups after training and did not reconstruct all split assignments. Supplementary precision, Brier score, and low-false-positive sensitivity are likewise labeled exploratory. Same-device exact repetition verifies the saved CUDA procedure, not portability across processor types (Supplementary Fig. S3 and Supplementary Tables S12–S13).

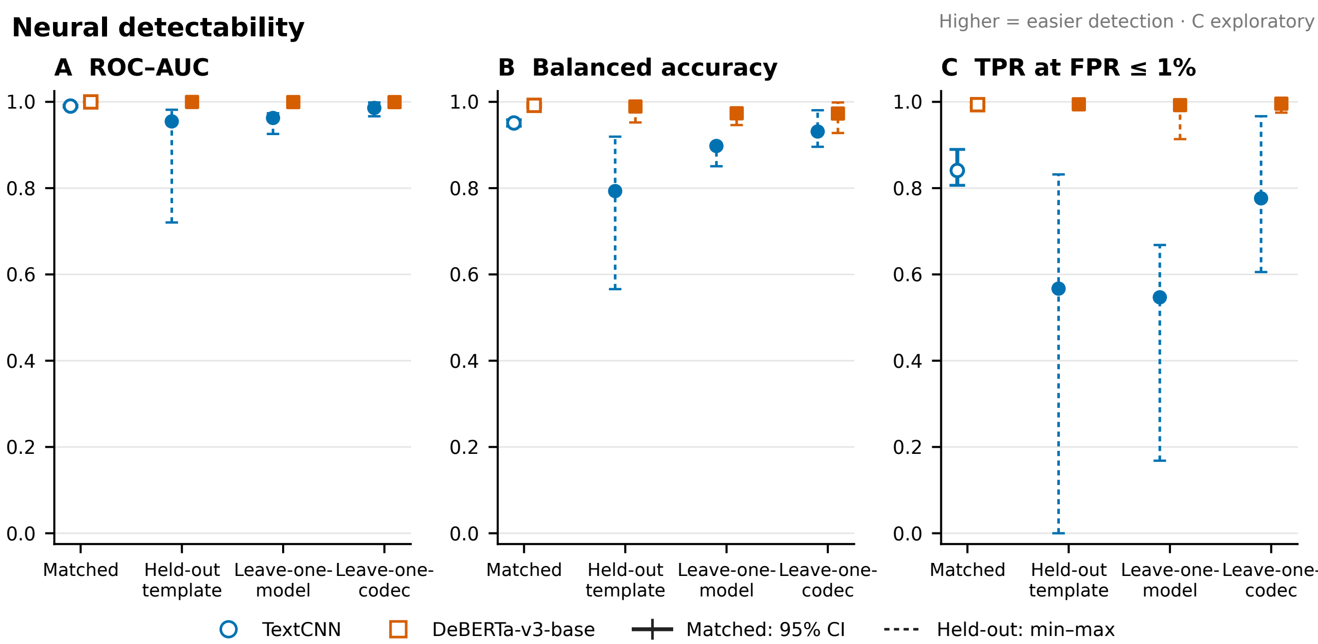

Figure 4. Neural detectability. Matched ROC-AUC and balanced accuracy are shown with payload-group bootstrap 95% confidence intervals; held-out-template, leave-one-model, and leave-one-codec results are summarized as ranges across frozen splits. Higher performance is adverse for RankCloak concealment because it indicates that covers are distinguishable from matched controls. Both classifiers completed all 28 regimes. Exact identity leakage checks passed, but a declared lexical near-duplicate diagnostic affected 16 splits; exclusion sensitivity produced small observed changes without eliminating that limitation. The full six-panel view and exploratory precision, Brier, low-false-positive, leakage, and repeatability results appear in Supplementary Figure S3.

Computational overhead and generalizability

Across the 18 primary model–protocol cells, mean inclusive generation time ranged from 1.674368 to 16.087628 seconds. Mean payload throughput ranged from 13.631855 to 100.851772 bits/s, and effective rate from 0.710004 to 7.322003 bits per full token. These saved wall times include wrapper setup and decoding scope defined by the harness; CPU time and repeated warm-up measurements were not retained.

The approximately order-of-magnitude spread across cells shows that representation and model choice materially affect operational cost in the retained environment. Compact direct ranks can reduce forced work, whereas byte expansion and repeated segmented continuations increase it. Throughput and effective rate answer different questions: the first normalizes payload bits by wall time, while the second normalizes by visible full-message tokens. Neither is a deployment benchmark outside the recorded wrapper and device.

On the fixed CUDA benchmark, representative TextCNN wall/training times were 91.358/9.498 seconds with 306.25 MB peak allocated VRAM; DeBERTa-v3-base times were 1,536.661/1,425.063 seconds with 6.38 GB. Those device-specific measurements do not generalize across hardware. Evaluation across three model families and exact-copy Spanish and Simplified Chinese trials broadens the controlled design, but does not validate very-large proprietary models, broad multilingual cover quality, operational channels, or real-world deployment.

The detector timings also illustrate the cost of the stronger adverse test: the pretrained architecture required substantially more saved time and memory than TextCNN in the single retained benchmark. CPU neural training was infeasible and was not used as a comparison point. These measurements support transparent study accounting, not a claim that the approach is efficient enough for an operational sender, receiver, or monitor (Supplementary Fig. S4 and Supplementary Tables S15–S16).

Discussion

RankCloak provides a reproducible method for converting serialized cryptographic artifacts into language-model next-token ranks and measuring the cover text that realizes them. Its representation choices expose a central trade-off. Direct subwords can be compact but leave rank pressure uncontrolled, whereas ASCII-byte and hex-nibble codecs bound the permitted rank by spending more forced positions. Segmentation redistributes those positions across contexts, and natural tails add non-payload length. The formal and empirical capacity relationships make these costs explicit rather than treating longer cover text as additional payload capacity.

Complete recovery in the evaluated primary and secondary-language matrices establishes deterministic codec and replay fidelity under the saved-token exact-copy contract. It does not establish an ordinary text channel. Detokenized visible text could change tokenization without an apparent edit, and common normalization, formatting, rewriting, or model-mismatch conditions sharply reduced or eliminated recovery. The contrast between perfect saved-token replay and fragile visible-text transmission is therefore not a secondary implementation detail; it defines the boundary of the recovery claim.

Cover-quality evidence was mixed and protocol-dependent. Aggregate automated readability values for ordinary controls and segmented full messages were similar, while source-model and held-out-evaluator contrasts changed direction across model and protocol cells. Forced spans and full messages also measure different mixtures of payload pressure and greedy continuation. These diagnostics help compare mechanisms, but they are not equivalent to human evaluation, and no participant evidence supports a claim of human-perceived naturalness.

Supporting analyses reinforce this interpretation without identifying a single optimum. Larger segments improved efficiency by reducing repeated tail overhead, while lead-ins worsened likelihood and length in the selected contrasts; removing tails mechanically improved rate but did not test perceived quality. These exploratory comparisons show that optimization depends on which cost is prioritized rather than establishing a universally preferable protocol. Evaluation across three model families, diverse prompts, and two secondary languages broadens the controlled setting, but cannot be extrapolated to arbitrary models, languages, or deployed channels.

Neural steganalysis supplies the clearest adverse finding. Both detector architectures achieved very high matched performance, and the pretrained detector remained highly effective across the evaluated held-out regimes. Although split sensitivity and lexical overlap limit extrapolation, the observed distinguishability directly argues against practical concealment in this corpus. Exact recovery cannot compensate for a cover distribution that trained classifiers can identify with near-perfect accuracy.

Despite these adverse results, the study contributes a controlled framework for separating representation, recovery assumptions, capacity, cover length, automated quality, transformation sensitivity, and detectability. The expanded payload, prompt, model, and secondary-language design shows how these quantities can be evaluated without equating model likelihood with reader judgment or deterministic replay with channel robustness. RankCloak therefore establishes a measurement framework and a set of reproducible failure surfaces for rank-based hiding; it does not establish a secure, undetectable, robust, or deployment-ready covert communication system.

Responsible use and limitations

RankCloak is dual use and can encode sensitive-looking artifacts, but it does not itself provide encryption, authentication, confidentiality, or a cryptographic-security proof; any protected payload still requires an external cryptographic system and an explicit threat model. Exact recovery depends on saved-token replay, whereas visible-text transmission and common transformations are fragile and the tested perturbations do not establish broad edit robustness. Human-perceived naturalness, broad multilingual behavior, real-world deployment, and performance across arbitrary future models or communication channels were not established; automated measures are not participant ratings, and detector performance is limited to the evaluated corpus. The evidence therefore supports RankCloak as a controlled research framework, not as a secure, undetectable, robust, or deployment-ready covert channel; detailed boundaries appear in Supplementary Table S17.

Data and Code availability

The RankCloak source code and computational data supporting this study are available at the GitHub repository, `https://github.com/mantzaris/llm-rankcloak`, and are archived in Zenodo at `https://doi.org/10.5281/zenodo.21987450`. The archive contains the public payload corpus, analysis inputs, detector predictions and metrics, source tables, figures, configurations, and provenance required to inspect the reported computational results.

References

1. Simmons, G. J. The prisoners' problem and the subliminal channel. In *Advances in Cryptology: Proceedings of CRYPTO '83*, 51–67 (Springer, 1984).
2. Cachin, C. An information-theoretic model for steganography. In *Information Hiding*, 306–318, DOI: 10.1007/3-540-49380-8_21 (Springer, 1998).
3. Hopper, N. J., Langford, J. & von Ahn, L. Provably secure steganography. In *Advances in Cryptology – CRYPTO 2002*, 77–92, DOI: 10.1007/3-540-45708-9_6 (Springer, 2002).
4. Petitcolas, F. A. P., Anderson, R. J. & Kuhn, M. G. Information hiding: A survey. *Proc. IEEE* **87**, 1062–1078, DOI: 10.1109/5.771065 (1999).
5. Wayner, P. Mimic functions. *Cryptologia* **16**, 193–214, DOI: 10.1080/0161-119291866883 (1992).
6. Chang, C.-Y. & Clark, S. Practical linguistic steganography using contextual synonym substitution and a novel vertex coding method. *Comput. Linguist.* **40**, 403–448, DOI: 10.1162/COLI_a_00176 (2014).
7. Fang, T., Jaggi, M. & Argyraki, K. Generating steganographic text with LSTMs. In *Proceedings of ACL 2017, Student Research Workshop*, 100–106, DOI: 10.18653/v1/P17-3017 (Association for Computational Linguistics, 2017).
8. Yang, Z.-L., Guo, X.-Q., Chen, Z.-M., Huang, Y.-F. & Zhang, Y.-J. RNN-Stega: Linguistic steganography based on recurrent neural networks. *IEEE Transactions on Inf. Forensics Secur.* **14**, 1280–1295, DOI: 10.1109/TIFS.2018.2871746 (2019).
9. Ziegler, Z. M., Deng, Y. & Rush, A. M. Neural linguistic steganography. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing*, 1210–1215, DOI: 10.18653/v1/D19-1115 (Association for Computational Linguistics, 2019).
10. Dai, F. Z. & Cai, Z. Towards near-imperceptible steganographic text. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, 4303–4308, DOI: 10.18653/v1/P19-1422 (Association for Computational Linguistics, 2019).
11. Shen, J., Ji, H. & Han, J. Near-imperceptible neural linguistic steganography via self-adjusting arithmetic coding. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, 303–313, DOI: 10.18653/v1/2020.emnlp-main.22 (Association for Computational Linguistics, 2020).
12. Zhang, S., Yang, Z., Yang, J. & Huang, Y. Provably secure generative linguistic steganography. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, 3046–3055, DOI: 10.18653/v1/2021.findings-acl.268 (Association for Computational Linguistics, 2021).
13. Wang, Y., Song, R., Li, L., Zhang, R. & Liu, J. Dynamically allocated interval-based generative linguistic steganography with roulette wheel. *Appl. Soft Comput.* **176**, 113101, DOI: 10.1016/j.asoc.2025.113101 (2025).
14. Wu, J., Wu, Z., Xue, Y., Wen, J. & Peng, W. Generative text steganography with large language model. In *Proceedings of the 32nd ACM International Conference on Multimedia*, 10345–10353, DOI: 10.1145/3664647.3680562 (Association for Computing Machinery, 2024).
15. Lin, K., Luo, Y., Zhang, Z. & Luo, P. Zero-shot generative linguistic steganography. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 5168–5182, DOI: 10.18653/v1/2024.naacl-long.289 (Association for Computational Linguistics, 2024).
16. Norelli, A. & Bronstein, M. M. LLMs can hide text in other text of the same length. In *The Fourteenth International Conference on Learning Representations* (2026).

17. Yan, R. & Murawaki, Y. Addressing tokenization inconsistency in steganography and watermarking based on large language models. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 7076–7098, DOI: 10.18653/v1/2025.emnlp-main.361 (Association for Computational Linguistics, 2025).

18. Wen, J., Zhou, X., Zhong, P. & Xue, Y. Convolutional neural network based text steganalysis. *IEEE Signal Process. Lett.* **26**, 460–464, DOI: 10.1109/LSP.2019.2895286 (2019).

19. Wu, H., Yi, B., Ding, F., Feng, G. & Zhang, X. Linguistic steganalysis with graph neural networks. *IEEE Signal Process. Lett.* **28**, 558–562, DOI: 10.1109/LSP.2021.3062233 (2021).

20. Wang, H., Yang, Z., Yang, J., Chen, C. & Huang, Y. Linguistic steganalysis in few-shot scenario. *IEEE Transactions on Inf. Forensics Secur.* **18**, 4870–4882, DOI: 10.1109/TIFS.2023.3298210 (2023).

21. Gehrmann, S., Strobelt, H. & Rush, A. GLTR: Statistical detection and visualization of generated text. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, 111–116, DOI: 10.18653/v1/P19-3019 (Association for Computational Linguistics, 2019).

22. Mitchell, E., Lee, Y., Khazatsky, A., Manning, C. D. & Finn, C. DetectGPT: Zero-shot machine-generated text detection using probability curvature. In *Proceedings of the 40th International Conference on Machine Learning*, vol. 202 of *Proceedings of Machine Learning Research*, 24950–24962 (PMLR, 2023).

23. Sadasivan, V. S., Kumar, A., Balasubramanian, S., Wang, W. & Feizi, S. Can AI-generated text be reliably detected? stress testing AI text detectors under various attacks. *Transactions on Mach. Learn. Res.* (2025).

24. Motwani, S. R. *et al.* Secret collusion among AI agents: Multi-agent deception via steganography. In *Advances in Neural Information Processing Systems*, vol. 37, 73439–73486 (Curran Associates, Inc., 2024).

25. Zolkowski, A., Nishimura-Gasparian, K., McCarthy, R., Zimmermann, R. S. & Lindner, D. Early signs of steganographic capabilities in frontier LLMs. In *The Fourteenth International Conference on Learning Representations* (2026).

26. National Institute of Standards and Technology. Secure hash standard (SHS). Federal Information Processing Standards Publication 180-4, National Institute of Standards and Technology (2015). DOI: 10.6028/NIST.FIPS.180-4.

27. Krawczyk, H., Bellare, M. & Canetti, R. HMAC: Keyed-hashing for message authentication. RFC 2104, DOI: 10.17487/RFC2104 (1997).

28. Dworkin, M. Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC. Special Publication 800-38D, National Institute of Standards and Technology (2007). DOI: 10.6028/NIST.SP.800-38D.

29. Nir, Y. & Langley, A. ChaCha20 and Poly1305 for IETF protocols. RFC 8439, DOI: 10.17487/RFC8439 (2018).

30. Josefsson, S. & Liusvaara, I. Edwards-curve digital signature algorithm (EdDSA). RFC 8032, DOI: 10.17487/RFC8032 (2017).

31. Josefsson, S. The base16, base32, and base64 data encodings. RFC 4648, DOI: 10.17487/RFC4648 (2006).

32. Davis, K., Peabody, B. & Leach, P. Universally unique IDentifiers (UUIDs). RFC 9562, DOI: 10.17487/RFC9562 (2024).

33. Wilson, E. B. Probable inference, the law of succession, and statistical inference. *J. Am. Stat. Assoc.* **22**, 209–212, DOI: 10.1080/01621459.1927.10502953 (1927).

34. Flesch, R. A new readability yardstick. *J. Appl. Psychol.* **32**, 221–233, DOI: 10.1037/h0057532 (1948).

35. Kim, Y. Convolutional neural networks for sentence classification. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing*, 1746–1751, DOI: 10.3115/v1/D14-1181 (Association for Computational Linguistics, 2014).

36. He, P., Liu, X., Gao, J. & Chen, W. DeBERTa: Decoding-enhanced BERT with disentangled attention. In *The Ninth International Conference on Learning Representations* (2021).

37. Efron, B. Bootstrap methods: Another look at the jackknife. *The Annals Stat.* **7**, 1–26, DOI: 10.1214/aos/1176344552 (1979).

38. Bates, D., Mächler, M., Bolker, B. & Walker, S. Fitting linear mixed-effects models using lme4. *J. Stat. Softw.* **67**, 1–48, DOI: 10.18637/jss.v067.i01 (2015).

39. Brooks, M. E. *et al.* glmmTMB balances speed and flexibility among packages for zero-inflated generalized linear mixed modeling. *The R J.* **9**, 378–400, DOI: 10.32614/RJ-2017-066 (2017).

Acknowledgements

None.

Author contributions

A.V.M. conceived the study, developed the methodology and software, conducted the original computational work, curated and interpreted the evidence, and is responsible for the manuscript.

Funding

No specific funding was reported.

Competing interests

The author declares no competing interests.